

```
/* ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
// ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
// ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
```

```
//
// LSU CSC 4103 Filesystem solution by Golden G. Richard III
// (@nolaforensix), 11/2017. Minor updates 11/2019. Updated 6/2022.
```

```
//
// This is a simple proof-of-concept solution that uses a fixed disk
// layout and inodes for block allocation tracking.
```

```
//
// This version is NOT thread-safe.
```

```
//
// ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
// ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
// ** DO NOT REDISTRIBUTE WITHOUT PERMISSION! **
```

```
//
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <limits.h>
#include "filesystem.h"
#include "softwaredisk.h"
```

```
//
// private filesystem constants. Real filesystems would dynamically
// determine these constants (and the resulting layout of the
// filesystem) based on the size of the storage device. These assume
// a software disk size of 4096 blocks. All of the values of these
// constants are interrelated--changing them arbitrarily will break
// everything.
//
```

```
#define MAX_FILES 512
#define DATA_BITMAP_BLOCK 0 // max SOFTWARE_DISK_BLOCK_SIZE*8 bits
#define INODE_BITMAP_BLOCK 1 // max SOFTWARE_DISK_BLOCK_SIZE*8 bits
#define FIRST_INODE_BLOCK 2
#define LAST_INODE_BLOCK 5 // 128 inodes per block.
// max of 4*128 = 512 inodes. thus 512 files
#define INODES_PER_BLOCK 128
#define FIRST_DIR_ENTRY_BLOCK 6
#define LAST_DIR_ENTRY_BLOCK 69
#define DIR_ENTRIES_PER_BLOCK 8
#define FIRST_DATA_BLOCK 70
#define LAST_DATA_BLOCK 4095
#define MAX_FILENAME_SIZE 507
#define NUM_DIRECT_INODE_BLOCKS 13
#define NUM_SINGLE_INDIRECT_BLOCKS (SOFTWARE_DISK_BLOCK_SIZE / sizeof(uint16_t))

#define MAX_FILE_SIZE (NUM_DIRECT_INODE_BLOCKS + NUM_SINGLE_INDIRECT_BLOCKS) * SOFTWARE_DISK_BLOCK_SIZE
```

```
//
// private filesystem types
//
```

```
// Type for one inode. Structure must be 32 bytes long.
```

```
typedef struct Inode {
    uint32_t size; // file size in bytes
    uint16_t b[NUM_DIRECT_INODE_BLOCKS+1]; // direct blocks + 1
```


// indirect block

} Inode:

```
1 // type for block of Inodes. Structure must be
2 // SOFTWARE_DISK_BLOCK_SIZE bytes (software disk block size).
3 typedef struct InodeBlock {
4     Inode inodes[SOFTWARE_DISK_BLOCK_SIZE / sizeof(Inode)];
5 } InodeBlock;
```

```
6
7 // type for one indirect block. Structure must be
8 // SOFTWARE_DISK_BLOCK_SIZE bytes (software disk block size). Max
9 // file size calculation is based on use of uint16_t as type for block
10 // numbers.
11 typedef struct IndirectBlock {
12     uint16_t b[NUM_SINGLE_INDIRECT_BLOCKS];
13 } IndirectBlock;
```

```
14
15 // type for one directory entry. Structure must be 512 bytes.
16 // Directory entry is free on disk if first character of filename is
17 // null.
18 typedef struct DirEntry {
19     uint8_t file_is_open; // file is open
20     int16_t inode_idx; // inode #
21     char name[MAX_FILENAME_SIZE]; // null terminated ASCII filename
22 } DirEntry;
```

```
23
24 // type for block of directory entries. Structure must be
25 // SOFTWARE_DISK_BLOCK_SIZE bytes (software disk block size).
26 typedef struct DirEntryBlock {
27     DirEntry direntry[SOFTWARE_DISK_BLOCK_SIZE / sizeof(DirEntry)];
28 } DirEntryBlock;
```

```
29
30 // type for a one block bitmap. Structure must be
31 // SOFTWARE_DISK_BLOCK_SIZE bytes (software disk block size).
32 typedef struct FreeBitmap {
33     uint8_t bytes[SOFTWARE_DISK_BLOCK_SIZE];
34 } FreeBitmap;
```

```
35
36 // main private file type
37 typedef struct FileInternals {
38     uint32_t position; // current file position
39     FileMode mode; // access mode for file
40     Inode inode; // cached inode
41     DirEntry d; // cached dir entry
42     int16_t d_index; // index number of cached dir entry
43 } FileInternals;
```

```
44
45 //
46 // private functions
47 //
```

```
48
49 // find index of first zero bit in a disk block. Returns -1 if no
50 // zero bit is found, otherwise returns the index of the zero bit.
51 static int32_t find_zero_bit(uint8_t *data) {
```

```
52     int32_t i, j;
53     int32_t freeidx = -1;
```

```
54
55     for (i=0; i < SOFTWARE_DISK_BLOCK_SIZE && freeidx < 0; i++) {
56         for (j=0; j <= 7 && freeidx < 0; j++) {
57             if ((data[i] & (1 << (7 - j))) == 0) {
```



```
    freeidx = i * 8 + j;
```

```
    }
```

```
return freeidx;
```

```
// find index of first free inode. Returns -1 if no inodes are free.  
// otherwise the index of the inode. Sets fserror on I/O error.
```

```
static uint16_t find_free_inode_idx(void) {
```

```
    FreeBitmap f;
```

```
    if (read_sd_block(&f, INODE_BITMAP_BLOCK)) {
```

```
        return find_zero_bit(f.bytes);
```

```
    }
```

```
    else {
```

```
        fserror = FS_IO_ERROR;
```

```
        return -1;
```

```
    }
```

```
// find first free data block. Returns -1 if no data blocks are free.  
// otherwise the block number of the free disk block. Sets fserror on  
// I/O error.
```

```
static int32_t find_free_block_idx(void) {
```

```
    FreeBitmap f;
```

```
    if (read_sd_block(&f, DATA_BITMAP_BLOCK)) {
```

```
        return find_zero_bit(f.bytes) + FIRST_DATA_BLOCK;
```

```
    }
```

```
    else {
```

```
        fserror = FS_IO_ERROR;
```

```
        return -1;
```

```
    }
```

```
// mark block 'blk' allocated (1) or free (0) according to 'flag'.  
// Returns 0 on error and 1 on success.
```

```
static uint32_t mark_block(uint16_t blk, uint32_t flag) {
```

```
    FreeBitmap f;
```

```
    blk -= FIRST_DATA_BLOCK;
```

```
    if (! read_sd_block(&f, DATA_BITMAP_BLOCK)) {
```

```
        return 0;
```

```
    }
```

```
    else {
```

```
        if (flag) {
```

```
            // set bit
```

```
            f.bytes[blk / 8] |= (1 << (7 - (blk % 8)));
```

```
        }
```

```
        else {
```

```
            // clear bit
```

```
            f.bytes[blk / 8] &= ~(1 << (7 - (blk % 8)));
```

```
        }
```

```
        if (! write_sd_block(&f, DATA_BITMAP_BLOCK)) {
```

```
            return 0;
```



```
    return 1;
```

```
}
```

```
// mark inode 'i' allocated (1) or free (0) depending on 'flag'.  
// Returns 0 on error and 1 on success. Sets fserror on I/O error.
```

```
static uint32_t mark_inode(uint16_t i, uint32_t flag) {
```

```
    FreeBitmap f;
```

```
    if (! read_sd_block(&f, INODE_BITMAP_BLOCK)) {
```

```
        fserror = FS_IO_ERROR;
```

```
        return 0;
```

```
    }
```

```
    else {
```

```
        if (flag) {
```

```
            // set bit
```

```
            f.bytes[i / 8] |= (1 << (7 - (i % 8)));
```

```
        }
```

```
        else {
```

```
            // clear bit
```

```
            f.bytes[i / 8] &= ~(1 << (7 - (i % 8)));
```

```
        }
```

```
        if (! write_sd_block(&f, INODE_BITMAP_BLOCK)) {
```

```
            fserror = FS_IO_ERROR;
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
// read inode with index 'idx'. Fills 'inode' if the index is valid.
```

```
// otherwise zeroes 'inode'. Returns 1 on success, 0 on failure.
```

```
// Sets fserror on I/O error.
```

```
static uint32_t read_inode(uint16_t idx, struct Inode *inode) {
```

```
    InodeBlock ib;
```

```
    uint16_t inodeblock = idx / sizeof(uint16_t) + FIRST_INODE_BLOCK;
```

```
    bzero(inode, sizeof(Inode));
```

```
    if (inodeblock > LAST_INODE_BLOCK) {
```

```
        return 0; // invalid inode index
```

```
    }
```

```
    else {
```

```
        if (read_sd_block(&ib, inodeblock)) {
```

```
            *inode = ib.inodes[idx % sizeof(uint16_t)];
```

```
            return 1;
```

```
        }
```

```
        else {
```

```
            fserror = FS_IO_ERROR;
```

```
            return 0;
```

```
        }
```

```
    }
```

```
}
```



```
// free all disk blocks associated with an inode with index 'i' and
// then free the inode. Returns 0 on failure and 1 on success. Sets
// fserror on I/O error.
```

```
static uint32_t free_inode(uint16_t i) {
    Inode inode;
    IndirectBlock id;

    int j;
    unsigned long freed=0;

    if (! read_inode(i, &inode)) {
        fserror = FS_IO_ERROR;
        return 0;
    }
    else {
        // free direct blocks
        for (j=0; j < NUM_DIRECT_INODE_BLOCKS; j++) {
            if (inode.b[j]) {
                if (! mark_block(inode.b[j], 0)) {
                    freed++;
                    fserror = FS_IO_ERROR;
                    return 0;
                }
            }
        }

        // free indirect blocks
        if (inode.b[NUM_DIRECT_INODE_BLOCKS]) {
            // read single indirect block
            if (! read_sd_block(&id, inode.b[NUM_DIRECT_INODE_BLOCKS])) {
                fserror = FS_IO_ERROR;
                return 0;
            }
            else {
                for (j=0; j < NUM_SINGLE_INDIRECT_BLOCKS; j++) {
                    if (id.b[j]) {
                        if (! mark_block(id.b[j], 0)) {
                            freed++;
                            fserror = FS_IO_ERROR;
                            return 0;
                        }
                    }
                }
            }
        }

        if (! mark_inode(i, 0)) {
            return 0;
        }
    }

    return 1;
}
```

```
// write inode with index 'idx'. Returns 1 on success, 0 on failure.
// Sets fserror on I/O error.
```

```
static uint32_t write_inode(uint16_t idx, struct Inode inode) {
    InodeBlock ib;

    uint16_t inodeblock = idx / sizeof(uint16_t) + FIRST_INODE_BLOCK;
    if (inodeblock <= LAST_INODE_BLOCK) {
```



```
if (! read_sd_block(&ib, inodeblock)) {  
    fserror = FS_IO_ERROR;  
    return 0;  
}
```

```
else {  
    ib.inodes[idx % sizeof(uint16_t)] = inode;  
    if (! write_sd_block(&ib, inodeblock)) {  
        fserror = FS_IO_ERROR;  
        return 0;  
    }  
}
```

```
return 1;  
}
```

```
// Returns the disk block number for the 'blocknum'th block in the  
// file associated with 'inode'. Returns -1 if 'blocknum' is invalid  
// or on an I/O error.
```

```
static int32_t get_inode_block_number(Inode inode, uint16_t blocknum) {
```

```
    IndirectBlock id;
```

```
    if (blocknum < NUM_DIRECT_INODE_BLOCKS) {
```

```
        // direct block
```

```
        return inode.b[blocknum];
```

```
    }  
    else if (! inode.b[NUM_DIRECT_INODE_BLOCKS]) {
```

```
        // empty single indirect block number, means all
```

```
        // single indirect blocks are zero
```

```
        return 0;
```

```
    }
```

```
    else if (blocknum - NUM_DIRECT_INODE_BLOCKS <=
```

```
        NUM_SINGLE_INDIRECT_BLOCKS) {
```

```
        // read single indirect block
```

```
        if (! read_sd_block(&id, inode.b[NUM_DIRECT_INODE_BLOCKS])) {
```

```
            return -1;
```

```
        }
```

```
        else {
```

```
            return id.b[blocknum - NUM_DIRECT_INODE_BLOCKS];
```

```
        }
```

```
    }
```

```
    else {
```

```
        // illegal block number
```

```
        return -1;
```

```
    }
```

```
// Sets the disk block number for the 'blocknum'th block in the cached  
// inode associated with file 'f'. Returns 0 on failure or 1 on  
// success. Sets fserror on error condition.
```

```
static int32_t set_inode_block_number(File f, uint16_t blocknum,  
                                       uint16_t newblock) {
```

```
    IndirectBlock id;
```

```
    int32_t new_id_block;
```

```
    if (blocknum < NUM_DIRECT_INODE_BLOCKS) {
```

```
        // direct block
```

```
        f->inode.b[blocknum] = newblock;
```



```

    }
    else if (blocknum - NUM_DIRECT_INODE_BLOCKS >
             NUM_SINGLE_INDIRECT_BLOCKS) {

```

```

        // illegal block number

```

```

        return 0;
    }

```

```

    else {

```

```

        if (! f->inode.b[NUM_DIRECT_INODE_BLOCKS]) {

```

```

            // inode doesn't have an indirect block yet

```

```

            bzero(&id, sizeof(IndirectBlock));

```

```

            new_id_block = find_free_block_idx();

```

```

            if (new_id_block < 0) {

```

```

                // failed to get an indirect block number

```

```

                return 0;
            }

```

```

            // mark new single indirect block allocated

```

```

            if (! mark_block(new_id_block, 1)) {

```

```

                fserror = FS_IO_ERROR;

```

```

                return 0;
            }

```

```

            // remember new single indirect block

```

```

            f->inode.b[NUM_DIRECT_INODE_BLOCKS] = new_id_block;
        }

```

```

    else {

```

```

        // read existing single indirect block

```

```

        if (! read_sd_block(&id, f->inode.b[NUM_DIRECT_INODE_BLOCKS])) {

```

```

            fserror = FS_IO_ERROR;

```

```

            return 0;
        }
    }

```

```

    // store new data block number in single indirect block

```

```

    id.b[blocknum - NUM_DIRECT_INODE_BLOCKS] = newblock;

```

```

    // now commit changes to the indirect block

```

```

    if (! write_sd_block(&id, f->inode.b[NUM_DIRECT_INODE_BLOCKS])) {

```

```

        fserror = FS_IO_ERROR;

```

```

        return 0;
    }

```

```

    // mark new data block allocated

```

```

    if (! mark_block(newblock, 1)) {

```

```

        fserror = FS_IO_ERROR;

```

```

        return 0;
    }

```

```

    // flush changes to inode

```

```

    if (! write_inode(f->d.inode_idx, f->inode)) {

```

```

        fserror = FS_IO_ERROR;

```

```

        return 0;
    }

```

```

    return 1;
}

```

```

// read data block 'blocknum' (indexed 0..#blocks-1) for file 'f',
// using the cached inode for mapping. 'data' must accomodate an

```



```
// entire disk block. Zero blocks are returned for blocks inside a
// file hole. Returns 1 on success and 0 on failure.
```

```
static uint32_t read_data_block(File f, uint16_t blocknum, void *data) {
    uint16_t datablock;
```

```
    datablock = get_inode_block_number(f->inode, blocknum);
```

```
    if (datablock >= FIRST_DATA_BLOCK &&
```

```
        datablock <= LAST_DATA_BLOCK) {
```

```
        if (! read_sd_block(data, datablock)) {
```

```
            fserror = FS_IO_ERROR;
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    else if (datablock == 0) {
```

```
        // file hole, return zero block
```

```
        bzero(data, SOFTWARE_DISK_BLOCK_SIZE);
```

```
    }
```

```
    else {
```

```
        // invalid block
```

```
        return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
// write data block 'blocknum' (0..#blocks-1) for file 'f', using the
// cached inode for mapping. 'data' must accomodate an entire disk
// block. Returns 1 on success and 0 on failure. Sets fserror only on
// an error condition.
```

```
static uint32_t write_data_block(File f, uint16_t blocknum, void *data) {
    uint16_t datablock;
```

```
    datablock = get_inode_block_number(f->inode, blocknum);
```

```
    if (datablock == 0) {
```

```
        // this block hasn't been allocated yet--either the file is being
```

```
        // extended or data is being written into a file hole, so need to
```

```
        // allocate a block
```

```
        // first find a free data block
```

```
        datablock = find_free_block_idx();
```

```
        if (datablock < 0) {
```

```
            // could also have failed because of an I/O error manipulating
```

```
            // the freespace bitmap, but we'll go with the more common
```

```
            // cause, which is no space left
```

```
            fserror = FS_OUT_OF_SPACE;
```

```
            return 0;
```

```
        }
```

```
        else {
```

```
            // mark new block allocated
```

```
            if (! mark_block(datablock, 1)) {
```

```
                fserror = FS_IO_ERROR;
```

```
                return 0;
```

```
            }
```

```
            // then add it to the inode (set_inode_block_number() commits
```



```

// changes to inode)
if (! set_inode_block_number(i, blocknum, datablock)) {
    // set_inode_block_number() sets fserror on error
    return 0;
}
}
}

```

```

if (datablock >= FIRST_DATA_BLOCK &&
    datablock <= LAST_DATA_BLOCK) {
    if (! write_sd_block(data, datablock)) {
        fserror = FS_IO_ERROR;
        return 0;
    }
}

```

```

else {
    // invalid block
    return 0;
}

```

```

return 1;
}

```

```

// locate a directory entry based on 'filename'. Returns index of the
// directory entry or -1 if the file can't be located. The directory
// entry 'd' is filled if a match is found, otherwise 'd' is zeroed.

```

```

static int32_t find_directory_entry(char *filename, DirEntry *d) {

```

```

    int32_t index = -1;

```

```

    int i, j;

```

```

    DirEntryBlock buf;

```

```

    for (i=FIRST_DIR_ENTRY_BLOCK; i <= LAST_DIR_ENTRY_BLOCK && index < 0; i++) {

```

```

        if (! read_sd_block(&buf, i)) {

```

```

            fserror = FS_IO_ERROR;

```

```

            break;

```

```

        }

```

```

        for (j=0; j < DIR_ENTRIES_PER_BLOCK && index < 0; j++) {

```

```

            if (! strncmp(filename, buf.dirent[j].name, MAX_FILENAME_SIZE-1)) {

```

```

                *d=buf.dirent[j];

```

```

                index = i * DIR_ENTRIES_PER_BLOCK + j;

```

```

            }

```

```

        }

```

```

    }

```

```

    if (index < 0) {

```

```

        bzero(d, sizeof(DirEntry));

```

```

    }

```

```

    return index;
}

```

```

// write directory entry 'd' at index 'index'. Returns 0 on failure
// and 1 on success.

```

```

static uint32_t write_directory_entry(DirEntry d, uint16_t index) {

```

```

    DirEntryBlock buf;

```

```

    // first read the block that contains the directory entry

```

```

    if (! read_sd_block(&buf, FIRST_DIR_ENTRY_BLOCK + index / DIR_ENTRIES_PER_BLOCK)) {

```

```

        fserror = FS_IO_ERROR;

```



```

    return 0;
}

buf.dirent[index % DIR_ENTRIES_PER_BLOCK] = d;

if (! write_sd_block(&buf, FIRST_DIR_ENTRY_BLOCK + index / DIR_ENTRIES_PER_BLOCK)) {
    fserror = FS_IO_ERROR;
    return 0;
}

```

```

    return 1;
}

```

```

// create a new directory entry 'd' on disk. Returns index of the new
// directory entry or -1 if an entry can't be created.

```

```

static int32_t create_directory_entry(DirEntry d) {
    int32_t index=-1;
    int i, j;
    DirEntryBlock buf;

    for (i=FIRST_DIR_ENTRY_BLOCK; i <= LAST_DIR_ENTRY_BLOCK && index < 0; i++) {
        if (! read_sd_block(&buf, i)) {
            fserror = FS_IO_ERROR;
            break;
        }
        for (j=0; j < DIR_ENTRIES_PER_BLOCK && index < 0; j++) {
            if (! buf.dirent[j].name[0]) {
                index = (i - FIRST_DIR_ENTRY_BLOCK) * DIR_ENTRIES_PER_BLOCK + j;
            }
        }
    }

    if (index >= 0 && ! write_directory_entry(d, index)) {
        index = -1;
    }

    return index;
}

```

```

// delete the directory entry 'd' located at 'index'. This is handled
// by freeing the associated inode and zeroing the directory entry.
// Returns 0 on failure and 1 on success.

```

```

static uint32_t delete_directory_entry(DirEntry d, uint16_t index) {

    free_inode(d.inode_idx);
    bzero(&d, sizeof(DirEntry));
    if (! write_directory_entry(d, index)) {
        fserror = FS_IO_ERROR;
        return 0;
    }
    else {
        return 1;
    }
}

```

```

//

```



```
// public functions
//
```

```
1 // open existing file with pathname 'name' and access mode 'mode'.
2 // Current file position is set to byte 0. Returns NULL on
3 // error. Always sets 'ferror' global.
4 File open_file(char *name, FileMode mode) {
```

```
5     File f = malloc(sizeof(FileInternals));
```

```
6
7     // filenames can't be NULL and they can't exceed maximum length
```

```
8     if (! name[0] || strlen(name) > MAX_FILENAME_SIZE - 1) {
```

```
9         ferror = FS_ILLEGAL_FILENAME;
```

```
10        goto err;
```

```
11    }
```

```
12
13    ferror = FS_NONE;
```

```
14    bzero(f, sizeof(FileInternals));
```

```
15    f->mode = mode;
```

```
16    f->position = 0;
```

```
17    ferror = FS_NONE;
```

```
18
19    f->d_index = find_directory_entry(name, &(f->d));
```

```
20    if (f->d_index < 0) {
```

```
21        ferror = FS_FILE_NOT_FOUND;
```

```
22        goto err;
```

```
23    }
```

```
24
25    // make sure file isn't already open
```

```
26    if (f->d.file_is_open) {
```

```
27        ferror = FS_FILE_OPEN;
```

```
28        goto err;
```

```
29    }
```

```
30
31    // mark file open
```

```
32    f->d.file_is_open = 1;
```

```
33    if (! write_directory_entry(f->d, f->d_index)) {
```

```
34        ferror = FS_IO_ERROR;
```

```
35        goto err;
```

```
36    }
```

```
37
38    // cache inode
```

```
39    if (! read_inode(f->d.inode_idx, &(f->inode))) {
```

```
40        ferror = FS_IO_ERROR;
```

```
41        goto err;
```

```
42    }
```

```
43
44    done:
```

```
45    return f;
```

```
46    err:
```

```
47    free(f);
```

```
48    return 0;
```

```
49 }
```

```
50
51 // create and open new file with pathname 'name' and (implied) access
```

```
52 // mode READ WRITE. Current file position is set to byte 0. Returns
```

```
53 // NULL on error. Always sets 'ferror' global.
```

```
54 File create_file(char *name) {
```

```
55 }
```



```

// filenames can't be NULL and they can't exceed maximum length
if (! name[0] || strlen(name) > MAX_FILENAME_SIZE - 1) {
    fserror = FS_ILLEGAL_FILENAME;
    goto err;
}

```

```

File f = malloc(sizeof(FileInternals));

```

```

fserror = FS_NONE;

```

```

bzero(f, sizeof(FileInternals));

```

```

f->d_index = find_directory_entry(name, &(f->d));

```

```

if (f->d_index >= 0) {

```

```

    // duplicate filenames aren't allowed

```

```

    fserror = FS_FILE_ALREADY_EXISTS;

```

```

    goto err;
}

```

```

f->mode = READ_WRITE;

```

```

f->position = 0;

```

```

fserror = FS_NONE;

```

```

// find and initialize an inode for the file

```

```

if ((f->d.inode_idx = find_free_inode_idx()) < 0) {

```

```

    fserror = FS_OUT_OF_SPACE;

```

```

    goto err;
}

```

```

// wipe inode and write it

```

```

bzero(&f->inode, sizeof(Inode));

```

```

if (! write_inode(f->d.inode_idx, f->inode)) {

```

```

    fserror = FS_IO_ERROR;

```

```

    goto err;
}

```

```

strncpy(f->d.name, name, MAX_FILENAME_SIZE-1);

```

```

f->d.name[MAX_FILENAME_SIZE-1] = 0;

```

```

f->d.file_is_open = 1;

```

```

if ((f->d_index = create_directory_entry(f->d)) < 0) {

```

```

    fserror = FS_OUT_OF_SPACE;

```

```

    goto err;
}

```

```

if (! mark_inode(f->d.inode_idx, 1)) {

```

```

    fserror = FS_IO_ERROR;

```

```

    goto err;
}

```

```

done:

```

```

return f;

```

```

err:

```

```

free(f);

```

```

return 0;
}

```

```

// close 'file'. Always sets 'fserror' global.

```

```

void close_file(File file) {

```

```

    fserror = FS_NONE;

```

```

    if (! file || ! file->d.file_is_open) {

```



```

    ferror = FS_FILE_NOT_OPEN;
}
else {
    file->d.file_is_open = 0;
    if (! write_directory_entry(file->d, file->d_index)) {
        ferror = FS_IO_ERROR;
    }
    free(file);
    file = NULL;
}
}

// read at most 'numbytes' of data from 'file' into 'buf', starting at
// the current file position. Returns the number of bytes read. If
// end of file is reached, then a return value less than 'numbytes'
// signals this condition. Always sets 'ferror' global.
unsigned long read_file(File file, void *buf, unsigned long numbytes) {
    unsigned long num_bytes_read = 0;
    uint16_t blocknum;
    uint32_t offset, tocopy;
    uint8_t block[SOFTWARE_DISK_BLOCK_SIZE];

    ferror = FS_NONE;
    if (! file || ! file->d.file_is_open) {
        ferror = FS_FILE_NOT_OPEN;
        goto done;
    }

    // limit read to current file size
    if (file->position + numbytes > file->inode.size) {
        numbytes = file->inode.size - file->position;
    }

    while (numbytes > 0) {
        if (! read_data_block(file,
                               file->position / SOFTWARE_DISK_BLOCK_SIZE,
                               block)) {
            ferror = FS_IO_ERROR;
            goto done;
        }

        offset = file->position % SOFTWARE_DISK_BLOCK_SIZE;
        tocopy = SOFTWARE_DISK_BLOCK_SIZE - offset > numbytes ?
            numbytes : SOFTWARE_DISK_BLOCK_SIZE - offset;
        memcpy(buf+num_bytes_read,
               block+offset,
               tocopy);
        numbytes -= tocopy;
        file->position += tocopy;
        num_bytes_read += tocopy;
    }

done:
    return num_bytes_read;
}

// write 'numbytes' of data from 'buf' into 'file' at the current file
// position. Returns the number of bytes written. On an out of space

```



```

// error, the return value may be less than 'numbytes'. Always sets
// 'fserror' global.
unsigned long write_file(File file, void *buf, unsigned long numbytes) {
1   unsigned long num_bytes_written = 0;
2   uint16_t blocknum;
3   uint32_t offset, tocopy;
4   uint8_t block[SOFTWARE_DISK_BLOCK_SIZE];
5
6   fserror = FS_NONE;
7   if (! file || ! file->d.file_is_open) {
8       fserror = FS_FILE_NOT_OPEN;
9       goto done;
10  }
11  else if (file->mode == READ_ONLY) {
12      fserror = FS_FILE_READ_ONLY;
13      goto done;
14  }
15  else if (file->position + numbytes > MAX_FILE_SIZE) {
16      fserror = FS_EXCEEDS_MAX_FILE_SIZE;
17      goto done;
18  }
19  else {
20      while (numbytes > 0) {
21          if (! read_data_block(file,
22                                file->position / SOFTWARE_DISK_BLOCK_SIZE,
23                                block)) {
24              fserror = FS_IO_ERROR;
25              goto done;
26          }
27
28          offset = file->position % SOFTWARE_DISK_BLOCK_SIZE;
29          tocopy = SOFTWARE_DISK_BLOCK_SIZE - offset > numbytes ?
30              numbytes : SOFTWARE_DISK_BLOCK_SIZE - offset;
31          memcpy(block + offset,
32                buf + num_bytes_written,
33                tocopy);
34          file->inode.size += tocopy;
35
36          // write_data_block() commits any changes to inode and also
37          // sets fserror on an error
38          if (! write_data_block(file,
39                                file->position / SOFTWARE_DISK_BLOCK_SIZE,
40                                block)) {
41              goto done;
42          }
43          numbytes -= tocopy;
44          file->position += tocopy;
45          num_bytes_written += tocopy;
46      }
47  }
48
49  done:
50      return num_bytes_written;
51  }
52
53
54  // sets current position in file to 'bytepos', always relative to the
55  // beginning of file. Seeks past the current end of file should
56  // extend the file. Returns 1 on success and 0 on failure. Always
57  // sets 'fserror' global.

```



```
int seek_file(File file, unsigned long bytepos) {
```

```
    ferror = FS_NONE;
```

```
    if (!file) {
```

```
        ferror = FS_FILE_NOT_OPEN;
```

```
        return 0;
```

```
    }
```

```
    else {
```

```
        // make sure bytepos makes sense--it has to be less than the maximum
```

```
        // file size to be legal
```

```
        if (bytepos < MAX_FILE_SIZE) {
```

```
            ferror = FS_NONE;
```

```
            // new position and potentially new file size. Don't allocate
```

```
            // space until there's a write, since we support file holes
```

```
            file->position = bytepos;
```

```
            if (file->position > file->inode.size) {
```

```
                file->inode.size = bytepos;
```

```
                // commit change to inode
```

```
                return write_inode(file->d.inode_idx, file->inode);
```

```
            }
```

```
        }
```

```
        else {
```

```
            ferror = FS_EXCEEDS_MAX_FILE_SIZE;
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
// returns the current length of the file in bytes. Always sets
```

```
// 'ferror' global.
```

```
unsigned long file_length(File file) {
```

```
    ferror = FS_NONE;
```

```
    return file->inode.size;
```

```
}
```

```
// deletes the file named 'name', if it exists. Returns 1 on success,
```

```
// 0 on failure. Always sets 'ferror' global.
```

```
int delete_file(char *name) {
```

```
    DirEntry d;
```

```
    int32_t index;
```

```
    ferror = FS_NONE;
```

```
    if ((index = find_directory_entry(name, &d)) < 0) {
```

```
        ferror = FS_FILE_NOT_FOUND;
```

```
        return 0;
```

```
    }
```

```
    else if (d.file_is_open) {
```

```
        ferror = FS_FILE_OPEN;
```

```
        return 0;
```

```
    }
```

```
    else {
```

```
        return delete_directory_entry(d, index);
```

```
    }
```

```
}
```



```
// determines if a file with 'name' exists and returns 1 if it exists,  
// otherwise 0. Always sets 'ferror' global.
```

```
int file_exists(char *name) {
```

```
    DirEntry d;
```

```
    ferror = FS_NONE;
```

```
    return find_directory_entry(name, &d) >= 0;
```

```
}
```

```
// describe current filesystem error code by printing a descriptive
```

```
// message to standard error.
```

```
void fs_print_error(void) {
```

```
    switch (ferror) {
```

```
    case FS_NONE:
```

```
        printf("No error.\n");
```

```
        break;
```

```
    case FS_OUT_OF_SPACE:
```

```
        printf("Disk space exhausted.\n");
```

```
        break;
```

```
    case FS_FILE_NOT_OPEN:
```

```
        printf("File not open.\n");
```

```
        break;
```

```
    case FS_FILE_OPEN:
```

```
        printf("File is open.\n");
```

```
        break;
```

```
    case FS_FILE_NOT_FOUND:
```

```
        printf("File not found.\n");
```

```
        break;
```

```
    case FS_FILE_READ_ONLY:
```

```
        printf("File access mode is read-only.\n");
```

```
        break;
```

```
    case FS_FILE_ALREADY_EXISTS:
```

```
        printf("File already exists.\n");
```

```
        break;
```

```
    case FS_EXCEEDS_MAX_FILE_SIZE:
```

```
        printf("Operation exceeds maximum file size.\n");
```

```
        break;
```

```
    case FS_ILLEGAL_FILENAME:
```

```
        printf("Illegal filename.\n");
```

```
        break;
```

```
    case FS_IO_ERROR:
```

```
        printf("I/O error. Something very bad happened.\n");
```

```
        break;
```

```
    default:
```

```
        printf("Unknown error %d.\n", ferror);
```

```
        break;
```

```
}
```

```
// filesystem error code set (set by each filesystem function)  
FSError ferror;
```

```
// extra function to make sure structure alignment on target platform  
// is correct. Returns 0 on failure or 1 on success.
```

```
int check_structure_alignment(void) {
```

```
    printf("Expecting sizeof(Inode) = 32, actual = %lu\n", sizeof(Inode));
```



```
printf("Expecting sizeof(IndirectBlock) = %d, actual = %lu\n", SOFTWARE_DISK_BLOCK_SIZE, sizeof(IndirectBlock));
printf("Expecting sizeof(InodeBlock) = %d, actual = %lu\n", SOFTWARE_DISK_BLOCK_SIZE, sizeof(InodeBlock));
printf("Expecting sizeof(DirEntry) = 512, actual = %lu\n", sizeof(DirEntry));
printf("Expecting sizeof(DirEntryBlock) = %d, actual = %lu\n", SOFTWARE_DISK_BLOCK_SIZE, sizeof(DirEntryBlock));
printf("Expecting sizeof(FreeBitmap) = %d, actual = %lu\n", SOFTWARE_DISK_BLOCK_SIZE, sizeof(FreeBitmap));
```

```
if (sizeof(Inode) != 32 ||
    sizeof(IndirectBlock) != SOFTWARE_DISK_BLOCK_SIZE ||
    sizeof(InodeBlock) != SOFTWARE_DISK_BLOCK_SIZE ||
    sizeof(DirEntry) != 512 ||
    sizeof(DirEntryBlock) != SOFTWARE_DISK_BLOCK_SIZE ||
    sizeof(FreeBitmap) != SOFTWARE_DISK_BLOCK_SIZE) {
```

```
    return 0;
```

```
}
```

```
else {
```

```
    return 1;
```

```
}
```